

Snake

System Description

Raymond Morrow

WSU Cpt S 322

Introduction

This document describes the system design and the functional requirements for the application we are making, the Snake Game. It outlines the target platform, users, interface structure, game states, game flow, storyboard transitions, and user stories. The goal of this is to define how this app will operate from both a technical and user perspective.

Snake is a single player arcade style game where the player controls a growing sprite, the snake, that moves around a grid based board. The objective is to collect food items to increase the snake's length while avoiding collisions with walls or the snake's growing body. The system must provide smooth controls, clear visual feedback, and reliable game states and transitions to ensure an engaging and responsive user experience.

System Description

The Snake game will target:

- Desktop Computers
- Keyboard input devices
- Built using a desktop compatible framework.

The game will be a windowed application and require minimal system resources.

Users

This section describes the types of users of the game.

- Player
 - A user who plays the game
 - Controls the snake movement
 - Attempts to achieve the highest score possible.
- Developer
 - Maintains and updates the game
 - Adjust the settings like speed or difficulty
 - Fixes bugs and improves gameplay mechanics.

User Interface Wireframes

This section describes the three main screens that the application will have.

- Main Menu
 - Title: Snake
 - Buttons:
 - Start Game
 - Settings
 - Exit
 - Displays High Scores
- Start of Game
 - Game Board
 - Snake appears on center on grid
 - First food item randomly generated

- Score Initialized to 0
- Game play
 - Snakes moves continuously
 - Food randomly appears
 - Score increases when food is eaten
 - Snake Grows in length
 - Boundaries visible
 - Real-Time Keyboard interactions
- Game Over
 - Triggered when
 - Snake hits the wall or collides with itself
 - Displays:
 - Final Score
 - High Score
 - Options:
 - Restart
 - Return to Main Menu
- Game Paused
 - Triggered by pressing P or Escape
 - Displays:
 - Resume
 - Restart
 - Main Menu
 - Move of the Snake Stops

Game States

List of the game states with descriptions

- Main Menu State: Waiting for users to start a game or adjust settings.
- Game Start State: Initializes game variables and board
- Play State: Active Gameplay, snake is in motion and user input is processed.
- Paused State: Game is momentarily stopped.
- Game Over State: Final score displayed, waiting for user action
- Settings State: User adjusts configuration options

Game Flow

Main Menu -> Start Game -> Initialize Game -> Playing -> (Paused Pressed) -> Paused -> Resume -> Playing
-> (Collision) -> Game Over
(Collision) -> Restart -> Initialize Game
(Collision) -> Main Menu

Game Storyboard

Show the transitions between screens for the game flow

Screen Transitions:

Main Menu

-> Start Game -> Game Play

Game Play

-> Pause -> Game Paused

-> Collision -> Game Over

Game Paused

-> Resume -> Game Play

-> Main Menu -> Main Menu

Game Over

-> Restart -> Game Play

-> Main Menu -> Main Menu

Setting

-> Main Menu

User Stories

List out user stories in the format

- As a player, I want to interact with the game board so I can control the snake and collect food.
- As a player, I want to start so that I can begin a new game session.
- As a player, I want to pause the game using a key press so that I can stop gameplay.
- As a player, I want to end the game by exiting so that I can close the application.
- As a player, I want to win by achieving a high score so that I feel accomplished.
- As a player, I want to lose if I collide with walls or myself so that the game maintains challenge.

1. Game Interaction

As a player

I want to control the snake's direction using arrow keys

So that I can navigate around the game board

****Acceptance Criteria:****

- Snake moves continuously in the current direction
- Arrow keys (Up, Down, Left, Right) change snake direction
- WASD keys also work for movement
- Snake cannot reverse 180 degrees (cannot go directly backwards)

Morrow, Raymond

- Movement is smooth and predictable
- Game speed increases slightly as snake grows

****Tasks:****

1. Implement Direction enumeration
2. Create keyboard input handler
3. Implement snake movement logic
4. Add direction validation (prevent 180-degree turns)
5. Test all directional inputs
6. Implement food spawning system
7. Implement score tracking
8. Perform integration testing across game states

****Priority:**** High

****Estimated Effort:**** 6-8 hours.